
PART 2 OF 5. WHERE MOST PEOPLE FIND OUT THEIR QUESTION WAS NEVER A QUESTION.

Turn the question into a rule

In part 1 you logged the sentence you type most weeks. Mine was some version of "is this still on track".

Here is the uncomfortable thing about that sentence. It is not a question. When I ask somebody whether their item is on track, I already know what would make me worried. The date has passed and nothing has changed. It is due Friday and it is still sitting in the same state it was in last Friday. I am not asking for information, I am running a rule in my head and asking them to confirm it.

That is good news, because a rule is something software can run and a question is not. This part is about writing yours down properly. There is still no code here. If you get this part wrong, everything you build afterwards will be confidently wrong, which is worse than having nothing.

1 Write the rule as an if, not as a feeling

Take your sentence and force it into this shape. Something is X if Y and Z.

Mine came out as two rules, not one. An item is late if its target date has passed and it is not in a closed state. An item needs a nudge if its target date falls within the next seven days and it has not changed state since the last time I looked.

Notice that both rules only use the three fields from part 1. Owner, state, date. If your rule needs a fourth thing, you either need to go get that field or your rule is doing something a person should be doing.

EXAMPLE

Write yours on one line, out loud, and make somebody on your team repeat it back. If they say "well, it depends", you have not finished writing it.

WHY THIS MATTERS: everything downstream is this sentence executed thousands of times. An hour here saves you from a tool that annoys people accurately.

2 Separate late from due soon from stuck

These feel like the same thing and they are three different conversations.

Late means the date has passed. The message is factual and slightly awkward for both of you, so it needs to be private and short.

Due soon means the date has not passed yet. This one is a favor, not a chase, and people respond to it completely differently. It is the highest value message you will send and the easiest one to get right.

Stuck means the state has not changed in a while, whatever the date says. Something in progress for six weeks is not on track just because its date is in December.

EXAMPLE

My split was late, due within seven days, and no state change in fourteen days. Three lists, three different messages, one query.

WHY THIS MATTERS: if you collapse these into one alert, everything reads like a telling off, and the first thing people do with a tool that tells them off is learn to ignore it.

3 Go and find your edge cases before they find you

Every tracker is full of items that will break your rule. Go looking for them now, on purpose.

Items with no date at all. Items assigned to somebody who left. Items in a state your rule has never heard of. Items closed but not marked closed. Parent items that look late because a child underneath them is late. Duplicates.

You do not have to handle all of these. You have to decide, in writing, what your rule does when it meets one. The usual right answer is to leave it out and put it on a list for a human.

EXAMPLE

Half my items had no target date. My rule now skips them and reports how many it skipped, every single run, so the gap stays visible instead of disappearing.

WHY THIS MATTERS: an automation that quietly drops things is worse than no automation, because you will start trusting it. Anything it cannot handle should be loud.

4 Write the query, then read it with your own eyes

Now you turn the rule into a query against whatever holds the work. The shape is the same everywhere even though the syntax is not.

In Jira, JQL looks like: `duedate < now() AND status != Done AND assignee is not EMPTY`.

In Azure DevOps, WIQL looks like: `SELECT Id FROM WorkItems WHERE [Target Date] < @Today AND [State] NOT IN ('Done', 'Closed', 'Removed')`.

In most other systems, including HSDes, it is a REST call with a filter on the same three fields, and the assistant you set up in part 1 can write it if you paste the API documentation and describe the rule in plain language.

EXAMPLE

Give it the rule exactly as you wrote it in step 1. My prompt was, here is our API documentation, write me a query that returns every open item where the target date has passed and the state is not closed, and print the owner, the title and how many days late it is.

WHY THIS MATTERS: the query is the cheap part. That is why it comes fourth instead of first.

5 Count the results and see whether you believe them

Run it. Do not send anything. Just look at the number.

If your rule says forty things are late and you know it is more like ten, your rule is wrong or your data is. If it says nothing is late and you spent yesterday chasing three people, the same applies. Either answer is useful and both of them arrive in about a minute.

Then read the actual list, not the count. Read every row. You are checking one thing: would you be comfortable if a colleague saw this exact list with their name on it.

EXAMPLE

The first time I ran mine it flagged an item as three months late that had actually shipped. Somebody had closed the work but not the parent. If I had been sending messages that day, I would have chased a senior engineer about finished work, and that is a mistake you only get to make once.

WHY THIS MATTERS: the whole system's credibility rests on the first messages being right. You get exactly one first impression with the people this touches.

You now have a rule you can defend, a list of things it deliberately ignores, and a query that produces something you believe.

Sit with the numbers for a few days before part 3. Run the query each morning and read the list. You are not building anything yet, you are finding out whether your tracker tells the truth, and almost nobody knows the answer to that about their own team.

Part 3 is next: getting the data properly, running it by hand for a week, and how to prompt a code assistant when you cannot write the code yourself.